# Independent Research Reflection

## Topic 1 — Java enums and mission states

### Why we chose this topic

Our simulator has a defined sequence of stages: pre-launch, ascent, reaching the target altitude, satellite deployment, and success or failure. We chose **enums** to represent these stages clearly and avoid managing mission progress through strings or several unrelated boolean variables.

### What we learned

An enum provides a fixed set of named, type-safe values. This makes comparisons and switch statements easier to read and reduces the chance of using an invalid state. We also learned that an enum **defines the available states but does not control transitions by itself**. The application still needs rules for when it may move from one state to another.

### How we applied it

In Mission.java, MissionState contains PRE_LAUNCH, ASCENT, ORBIT, DEPLOYMENT, SUCCESS, and FAILURE. The methods launch(), checkOrbitReached(), deploySatellite(), completeMission(), and failMission() enforce the mission's progression. A separate MissionEvent enum defines the four possible emergencies. MissionController handles these events, while MissionPane uses the current mission state to display its status and enable appropriate buttons. For example, satellite deployment is only available after the target-altitude stage is reached.

### Challenges, decisions and impact

The main decision was to **separate overall mission progress from temporary emergency events**. An overheating warning, for example, should not replace the fact that the rocket is in ascent. We also kept state transitions in Mission rather than changing them directly from the GUI. This made the program flow easier to understand and helped prevent invalid actions, such as deploying the satellite before the mission reaches the required stage.

## Topic 2 — Multithreading with ScheduledExecutorService

### Why we chose this topic

We wanted **rocket physics, mission-state checks, and event detection to run separately from the JavaFX interface and animations**. The flight must progress continuously while telemetry is displayed and the player responds to emergencies. Running the simulation on a background worker keeps those calculations off the JavaFX Application Thread, which is responsible for drawing and user interaction.

### What we learned and why we chose ScheduledExecutorService

A regular Java Thread can run the simulation, but **we would have to implement much of the repeated-update behavior ourselves**: write a loop in run(), call Thread.sleep() between updates, handle interruptions, manage a stop condition, and arrange cleanup when the mission finishes. To maintain a consistent update rate, we would also need to account for the time spent performing each physics calculation; simply sleeping for 50 ms after every update makes the real interval longer than 50 ms. We chose **ScheduledExecutorService because these scheduling and task-management facilities are built in**. scheduleAtFixedRate() runs the update repeatedly at the requested interval without a custom timing loop, a ScheduledFuture lets us cancel subsequent executions with cancel(false), and the executor can be shut down when it is no longer needed. Executors.newSingleThreadScheduledExecutor() also ensures that simulation updates run one at a time. The executor still uses a background thread internally, but **manages its scheduling and lifecycle for us**, making the code shorter and easier to maintain. We also learned that JavaFX controls must be updated on the JavaFX Application Thread. Therefore, the simulation worker sends interface changes through Platform.runLater() instead of modifying controls directly.

### How we applied it

MissionController creates the executor using Executors.newSingleThreadScheduledExecutor(). After the ignition animation, it schedules updateSimulation() every **50 milliseconds**, corresponding to the simulator's 0.05-second time step. The worker updates the Rocket model, checks mission progress and emergencies, and gathers altitude, velocity, fuel, and throttle values. Platform.runLater() then updates the telemetry, rocket display, and mission status on the JavaFX thread. The animations remain part of the JavaFX view, not the physics loop.

### Challenges, decisions and impact

We had to manage **communication between the worker and the GUI thread**. Rocket uses synchronized methods to protect shared flight data, and volatile is used for flags that must be visible across threads. At the end of a flight, simulationTask.cancel(false) stops future scheduled updates; shutdown() also closes the executor when the controller is disposed of. These built-in controls avoided writing our own repeating-thread management and helped keep the interface responsive during a mission.